

Nonlinear Temporal Reasoning Thesis Blueprint

Thesis Blueprint: Nonlinear Temporal Reasoning in LLMs — Beyond Memorized Chronology

1. Motivation

Sun et al. (2026) benchmark LLMs on sentence-level event ordering using shuffled, single-chronology Wikidata biographies under absolute-time (AT) and mixed-time (MT) settings. Two gaps remain:

1. **“Nonlinear” is not actually nonlinear.** Their task is a single, real, linear chronology presented out of order — one entity, one timeline, sentences scrambled. True nonlinear narrative (multiple interleaved timelines, flashback/frame structure, fiction with non-chronological plotting) is untested.
2. **Memorization confound.** Their source data is real Wikidata biographies of real, often notable people. A model doesn’t need to reason over the *given* text at all if it already “knows” (from pretraining) that Einstein published special relativity in 1905 and won the Nobel Prize in 1921 — it can just recall the correct order. High scores under AT could reflect **retrieval of memorized facts**, not temporal reasoning over the passage in front of it.

This thesis targets both gaps together: build a **nonlinear, multi-thread narrative benchmark** (including fiction) and a **counterfactual/perturbed variant** (altered names and dates) that forces models to reason from the text alone, then test whether a **temporal-graph-based pipeline** narrows the resulting performance gap.

2. Research Questions

- **RQ1:** How much worse do LLMs perform on genuinely nonlinear narratives (multi-thread, flashback, fiction) compared to the single-chronology shuffled setting in prior work?
- **RQ2:** How much of LLMs’ apparent temporal reasoning ability is actually memorized world knowledge, versus reasoning over the given passage? (Tested via name/date perturbation.)
- **RQ3:** Does decomposing the task into an explicit temporal graph (extract nodes/edges, then sort) improve robustness on nonlinear and perturbed narratives compared to direct one-shot ordering prompts?

3. Hypotheses

- **H1:** Model accuracy drops substantially more on multi-thread/flashback passages than on single-thread shuffled passages

of the same event count, because thread attribution is an added reasoning step the original task never requires.

- **H2:** Performance drops significantly on the perturbed (fictionalized name/date) version of the *same* passages relative to the original — the size of this drop estimates how much of the original score was memorization rather than reasoning.
- **H3:** A graph-mediated pipeline (extract events + relations → build graph → topological sort) degrades more gracefully than direct prompting on both the nonlinear and perturbed conditions, because it separates thread attribution, relation extraction, and sorting into smaller, individually-checkable steps instead of one opaque pass.
- **H3a (diagnostic):** Most of the remaining failure in the graph pipeline will localize to thread attribution and cross-thread alignment, not to local relation extraction or sorting — identifying *where* nonlinear reasoning breaks down, not just *that* it does.

4. Worked Example: A Nolan-Style Interleaved Timeline

Films like *Dunkirk* interleave several threads that move through time at different rates and only converge at the end; *Memento* runs one thread forward and another backward simultaneously. This is the kind of structure the original paper's single-chronology setup can never produce, and it's exactly the failure mode worth benchmarking. Below is a mock passage in that spirit — three threads, different granularities, presented out of order, the way a model would actually receive it.

Threads (ground truth, hidden from the model): - **Thread L (weeks):** a land-based storyline spanning several weeks - **Thread S (a single day):** a sea-based storyline spanning one day - **Thread A (a single hour):** an air-based storyline spanning one hour - All three threads converge at the same final event.

Shuffled input passage (what the model sees):

1. Farrow scans the horizon from the cockpit, fuel gauge dropping fast.
2. Tanner has been dug into the sand for six days, waiting for a ship that never comes.
3. Dawson pushes his fishing boat out from the coast at first light, three hours from the beach.
4. Farrow banks hard as a plane goes down below him, the fight barely a minute old.
5. Tanner watches the mole get bombed for the third time that week.
6. Dawson pulls a shivering pilot from the water, radio crackling with news from the beach.
7. Farrow, gliding now with the engine cut, sees hundreds of men lined up on the sand below.
8. Tanner finally boards a ship, one week after he first reached the beach.
9. Dawson's boat reaches the mole just as Tanner's ship is pulling away.

Gold structure the model must recover (not shown to it): -

Thread L (Tanner): 2 → 5 → 8 - Thread S (Dawson): 3 → 6 → 9 - Thread A (Farrow): 4 → 1 → 7 - Cross-thread convergence point: sentences 8 and 9 depict the same moment from two threads.

Why this is harder than the original benchmark's task: - The model must first work out *which thread each sentence belongs to* (there's no explicit entity name pattern like "Person X, born in 1921" to key off — thread identity has to be inferred from the recurring character and setting). - Each thread has its own internal granularity (weeks / a day / an hour) that don't share a common absolute clock — there is no single "1945" to sort everything by. - Some sentences must be recognized as depicting the **same moment** across threads (8 and 9) rather than being sorted as sequential events, which the original task's permutation-only framing has no way to express or score.

Perturbed / counterfactual version (for the memorization test): the same passage with the character names and any real-world-anchored details replaced — e.g. "Tanner" → "Corriell", "Dawson" → "Halvorsen" — and, for the real-biography-derived subset of the dataset, dates shifted by a random offset per passage. Since this example is already fictional, it mainly illustrates the *structure* the real perturbation pipeline needs to preserve: relative order and granularity intact, no real-world fact left for a model to recall instead of reason through.

5. Dataset Construction

Reuse and extend the original paper's Wikidata extraction pipeline (their GitHub release) rather than building from scratch. Three subsets:

A. Multi-thread nonlinear set - Interleave 2 (or, per the worked example, up to 3) entities' or storylines' event sentences into one shuffled passage, optionally at different temporal granularities as above. - Task: recover each thread's individual order, plus a merged global order and any cross-thread convergence points.

B. Flashback/frame set - Take single-entity passages and deliberately insert a later event early, using framing language ("years before this, X had already...", "looking back on 1998..."). - Task: recover true chronology despite narrative-order cues designed to mislead.

C. Perturbed / counterfactual set (paired with A and B) - For a subset of passages in A and B, replace real entity names with fictional ones and shift all dates by a random per-passage offset (preserving relative gaps/order, so the gold permutation is unchanged) or substitute plausible-but-fictional dates entirely. - This is the memorization probe: same reasoning difficulty, but no real-world fact to recall. - Optionally add a small hand-authored **fiction** slice (invented characters/events from the start, not perturbed real data, similar in spirit to the worked example above) as a second, independent check on H2.

6. Method: Temporal-Graph Pipeline

1. **Node extraction + thread attribution:** LLM extracts each event as a node tagged with its entity/thread and any time expression (absolute, relative, or flashback marker).
2. **Edge construction:** LLM extracts local pairwise relations only ("before/after X", "N years after Y", "same period as Z", "same moment as W") — a shallower, more checkable task than global ordering.
3. **Per-thread topological sort:** deterministic graph algorithm, not the LLM — removes the weakest link (generating long correct

- permutations) from the LLM’s job.
4. **Cross-thread alignment:** merge threads using shared anchors (absolute dates where available, or explicit convergence points like sentences 8/9 above).

Baseline for comparison: the original paper’s one-shot direct-prompting method, run on the same new datasets.

7. Evaluation

- Reuse original metrics: Exact Match, Kendall’s τ , pairwise accuracy.
- Add: **thread-attribution accuracy** (did the model assign each event to the right thread), **convergence-point detection** (did the model recognize same-moment events across threads), **stage-wise accuracy** for the graph pipeline (node extraction, edge correctness, final sort), and **memorization gap** (original-data score minus perturbed-data score, per model/condition).
- Report all results against the original paper’s published numbers as the linear/AT reference point, so degradation curves are directly comparable.

8. Expected Contributions

1. First benchmark isolating **genuine nonlinear narrative structure** (multi-thread, flashback, fiction) from simple shuffled-presentation order, directly extending a known gap in prior work.
2. First empirical estimate of **how much of LLM temporal-ordering performance is memorization** of real-world biographical facts vs. text-grounded reasoning, via matched original/perturbed passage pairs.
3. A **diagnostic, stage-wise graph pipeline** that not only tests whether structure helps, but localizes *where* nonlinear reasoning fails (attribution vs. relation extraction vs. sorting vs. cross-thread merge) — something the original paper’s single EM number cannot do.

9. Risks / Fallbacks

- If multi-thread interleaving proves too complex to construct at scale, fall back to flashback-only (Set B) as the core nonlinear contribution.
- If the graph pipeline underperforms direct prompting, that is still a valid and reportable negative result — pair it with the stage-wise diagnostic to explain *why*.
- If perturbation shows little memorization gap, that’s also a real finding (suggests genuine reasoning, not recall) — frame H2 as a falsifiable test either way, not a foregone conclusion.